

GUIA PRACTICA OFICIAL · SESION 1

LABORATORIO 0 CONFIGURACION Y “HOLA FLUTTER”

Ejecutar · Modificar · Evidenciar · Versionar

La primera victoria tecnica del Modulo 3

Mision del laboratorio: terminar la sesion con Flutter operativo, un dispositivo detectado, la primera aplicacion ejecutandose, una modificacion visible mediante *hot reload* y un repositorio con evidencia verificable.

RESULTADO 1

ENTORNO OPERATIVO

Flutter y Android listos

RESULTADO 2

APP EJECUTANDOSE

Emulador o telefono

RESULTADO 3

PRIMER COMMIT

Historial iniciado

Ficha de trabajo

Sesion	Sesion 1: panorama movil, arquitectura y configuracion del entorno.
Laboratorio	Lab 0: configuracion del entorno y primera aplicacion Flutter.
Duracion	110 minutos de laboratorio guiado mas trabajo no lectivo.
Modalidad	Individual, con apoyo colaborativo controlado.
Tecnologia	Flutter SDK, Dart, VS Code o Android Studio, Git y Android.
Producto	Proyecto Flutter ejecutable, personalizado y versionado.
Evidencia central	Ejecucion real, capturas, repositorio y explicacion breve.

Objetivo

Configurar o validar un entorno profesional de desarrollo movil, ejecutar una primera aplicacion Flutter en un emulador o dispositivo fisico, comprobar el *hot reload* e iniciar el control de versiones del proyecto.

Criterios de exito

El laboratorio se considera logrado cuando:

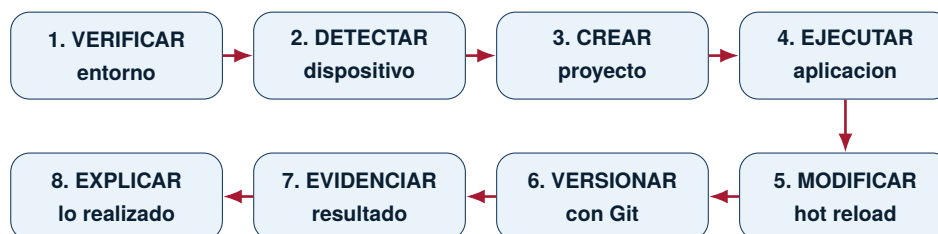
- Flutter responde desde la terminal.
- Existe un dispositivo disponible.
- La aplicacion se ejecuta sin error critico.
- Se observa una modificacion en caliente.

La evidencia queda completa cuando:

- el repositorio contiene commits progresivos;
- las capturas estan identificadas;
- el README explica el entorno usado;
- el estudiante puede repetir el proceso.

Regla profesional: una captura aislada no demuestra dominio. La evidencia valida combina ejecucion, codigo, historial y explicacion.

Ruta del laboratorio



Nombre recomendado del proyecto: use minúsculas, números y guion bajo; no utilice espacios, tildes ni caracteres especiales.

hola_flutter_apellido

Fase 1. Verificación técnica inicial

Abra una terminal nueva. No reutilice una terminal abierta antes de configurar el PATH, porque podría conservar valores antiguos.

```
flutter --version
flutter doctor
flutter devices
git --version
```

Interpretación rápida

Comando	Que debe comprobar	Resultado mínimo
flutter --version	SDK reconocido y versión visible.	OPERATIVO
flutter doctor	Estado de Flutter, Android y editor.	Sin error crítico relevante.
flutter devices	Teléfono, emulador o destino disponible.	Al menos un dispositivo.
git --version	Git accesible desde terminal.	Versión identificada.

Importante: flutter doctor puede mostrar advertencias de componentes que no se utilizarán en esta sesión, por ejemplo iOS en Windows. La prioridad es dejar operativos Flutter, Android Toolchain y el dispositivo elegido.

Punto de control 1

- ☐ Flutter responde en terminal.
- ☐ Git responde en terminal.
- ☐ Android Toolchain no presenta bloqueo crítico.
- ☐ Existe teléfono, emulador o plan de contingencia.

Fase 2. Preparación del dispositivo

OPCION A · TELEFONO FISICO

1. Active las opciones de desarrollador.
2. Habilite la depuracion USB.
3. Conecte un cable que transfiera datos.
4. Acepte la autorizacion RSA.
5. Ejecute `flutter devices`.

OPCION B · EMULADOR ANDROID

1. Abra Device Manager en Android Studio.
2. Cree o inicie un dispositivo virtual.
3. Espere hasta visualizar Android.
4. Ejecute `flutter devices`.
5. Compruebe que aparece el emulador.

```
flutter devices
flutter emulators
# Si conoce el identificador:
flutter emulators --launch <id_del_emulador>
```

No continúe a ciegas. Si el dispositivo no aparece, resuelva primero cable, permisos, emulador, controlador o autorización.

Plan de contingencia

1. Usar telefono fisico antes que instalar un emulador durante la clase.
2. Utilizar un emulador previamente configurado por el docente.
3. Ejecutar temporalmente en Chrome para verificar proyecto y *hot reload*.
4. Trabajar en pareja durante la sesion y completar individualmente como HNL.

Fase 3. Creacion del proyecto

Ubique la terminal en una carpeta de trabajo sencilla. Evite rutas largas, espacios innecesarios o carpetas sincronizadas que puedan bloquear archivos.

```
flutter create hola_flutter_apellido
cd hola_flutter_apellido
```

Ejemplo valido: `hola_flutter_cruz`.

No recomendados: `Hola Flutter`, `Mi Aplicacion`, `practica-nro-1`.

Reconocimiento de la estructura

Elemento	Funcion inicial
lib/main.dart	Punto de entrada y codigo principal.
pubspec.yaml	Dependencias, recursos y configuracion.
android/	Archivos especificos de Android.
ios/	Archivos para iOS; compilacion requiere macOS.
test/	Pruebas automatizadas.
README.md	Descripcion e instrucciones de ejecucion.

Mensaje clave: hoy no necesitamos comprender cada carpeta. Necesitamos reconocer donde esta el codigo principal, donde se declaran dependencias y donde se documenta el proyecto.

Fase 4. Primera ejecucion

Con el dispositivo activo, ejecute:

```
flutter run
```

Si existen varios dispositivos:

```
flutter devices
flutter run -d <identificador_del_dispositivo>
```

Teclas durante la ejecucion

Tecla	Accion
r	<i>Hot reload:</i> aplica cambios conservando estado cuando es posible.
R	<i>Hot restart:</i> reinicia desde el estado inicial.
q	Finaliza la ejecucion controladamente.
h	Muestra ayuda.

Punto de control 2

Registre la aplicacion inicial antes de modificarla: evidencias/03_app_original.png.

Fase 5. Personalizacion y hot reload

Abra lib/main.dart y realice modificaciones controladas.

Cambios obligatorios

1. Cambie el titulo de la aplicacion.
2. Modifique el texto central.
3. Cambie el color principal mediante `colorSchemeSeed`.
4. Agregue su nombre o identificacion del laboratorio.
5. Guarde y observe el cambio mediante *hot reload*.

Si prefiere reemplazar temporalmente el contenido por una version controlada, utilice este ejemplo:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MiPrimeraApp());
}

class MiPrimeraApp extends StatelessWidget {
  const MiPrimeraApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Laboratorio 0',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF173A63),
        ),
        useMaterial3: true,
      ),
      home: const PantallaInicio(),
    );
  }
}

class PantallaInicio extends StatelessWidget {
  const PantallaInicio({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Mi primera app Flutter'),
      ),
      body: const Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            Icon(Icons.rocket_launch, size: 72),
            SizedBox(height: 16),
            Text(
              'Flutter esta funcionando',
              style: TextStyle(
                fontSize: 24,
```

```
fontWeight: FontWeight.bold,  
)  
)  
  SizedBox(height: 8),  
  Text('Laboratorio 0 - Modulo 3'),  
],  
)  
)  
)  
);  
  
}  
  
}
```

No copie sin observar. Identifique al menos cinco widgets: MaterialApp, Scaffold, AppBar, Column, Text e Icon.

Reto de personalizacion

Nivel	Reto	Evidencia
Base	Cambiar textos, color y nombre.	Captura actualizada.
Pro	Agregar icono y reorganizar contenido.	Ejecucion sin desborde.
Epico	Incorporar un boton con SnackBar.	Demo en vivo.

Punto de control 3

- ☐ La app conserva ejecucion estable.
- ☐ El cambio se visualizo sin reinstalar manualmente.
- ☐ El estudiante indica que archivo modifiko.
- ☐ Existe captura posterior al cambio.

Fase 6. Primer historial con Git

Desde la raiz del proyecto:

```
git status
```

Si todavia no es un repositorio:

```
git init
```

Registre el avance en dos pasos:

```
git add .  
git commit -m "chore: crea proyecto Flutter inicial"
```

```
git add .  
git commit -m "feat: personaliza primera pantalla Flutter"
```

Por que dos commits? Porque el historial debe mostrar evolucion. Un unico commit final oculta el proceso.

Repositorio remoto

1. Cree m3_flutter_nombre_apellido.
2. No agregue otro README si el proyecto ya contiene uno.
3. Vincule el repositorio local.
4. Realice el primer push.

```
git remote add origin <URL_DEL_REPOSITORIO>  
git branch -M main  
git push -u origin main
```

Nunca suba claves, contraseñas, tokens, archivos .env ni credenciales.

Evidencias y estructura de entrega

Cree una carpeta de evidencias:

```
mkdir evidencias
```

```
hola_flutter_apellido/  
|-- lib/  
|-- evidencias/  
|   |-- 01_flutter_doctor.png  
|   |-- 02_dispositivo_detectado.png  
|   |-- 03_app_original.png  
|   '-- 04_app_personalizada.png  
|-- README.md  
'-- pubspec.yaml
```

README minimo

```
# Laboratorio 0 - Hola Flutter  
  
## Estudiante  
Nombre completo:  
  
## Entorno
```

```

Sistema operativo:
Version de Flutter:
Dispositivo utilizado:

## Resultado
La aplicacion fue creada, ejecutada y personalizada.

## Dificultad principal
Describir el problema y como fue resuelto.

## Evidencias
Incluir las capturas de la carpeta evidencias.

```

Lista final de entrega

N.	Evidencia	Estado
1	flutter doctor sin bloqueo critico relevante.	☐
2	Dispositivo visible mediante flutter devices.	☐
3	Aplicacion original ejecutandose.	☐
4	Aplicacion personalizada mediante hot reload.	☐
5	Repositorio con al menos dos commits.	☐
6	README y carpeta de evidencias actualizados.	☐
7	Explicacion oral breve del proceso.	☐

Matriz de intervencion rapida

Sintoma	Diagnostico probable	Primera accion
flutter no reconocido	PATH no actualizado o SDK mal ubicado.	Abrir terminal nueva y revisar ruta.
Android Toolchain con error	SDK, herramientas o licencias pendientes.	Revisar flutter doctor -v.
Telefono no aparece	Cable, depuracion o autorizacion.	Cambiar cable y aceptar RSA.
Dispositivo unauthorized	Autorizacion USB rechazada.	Revocar permisos y reconectar.
Emulador lento	Virtualizacion o recursos insuficientes.	Usar telefono fisico.
Gradle no descarga	Internet, proxy o bloqueo temporal.	Verificar red y reintentar.
La app no refleja cambios	Archivo no guardado o recarga incorrecta.	Guardar y presionar r.
Git rechaza el commit	Nombre o correo no configurados.	Configurar identidad de Git.

```

git config --global user.name "Nombre Apellido"
git config --global user.email "correo@example.com"

```

Cultura de depuracion: lea el mensaje, identifique el componente, pruebe una accion controlada y registre la solucion.

Verificacion formativa

Esta tabla orienta la retroalimentacion y no reemplaza la ponderacion oficial del modulo.

Criterio	Valor	Logro esperado
Entorno validado	20	Flutter y Git responden sin bloqueo relevante.
Dispositivo detectado	15	Telefono o emulador disponible.
Aplicacion ejecutada	25	Proyecto inicia y funciona.
Personalizacion y hot reload	20	Cambio propio visible y explicado.
Git, README y evidencias	20	Historial progresivo y entrega organizada.
TOTAL	100	Laboratorio completo y defendible.

Trabajo no lectivo

1. Completar cualquier componente pendiente.
2. Personalizar la aplicacion con identidad propia.
3. Subir el repositorio a GitHub.
4. Actualizar README y evidencias.
5. Revisar variables, funciones y tipos de datos en Dart.

RESULTADO DE LA SESION

El participante no termina diciendo "instale Flutter". Termina demostrando:

"Cree, ejecute, modifique y versione mi primera aplicacion Flutter."

M.Sc. Ing. Juan Pablo Cruz Ovando

Docente responsable

Modulo 3 · Desarrollo de Aplicaciones Moviles